

ITMO | 2026

Практика 03: Разработка с помощью AI IDE

Прежде чем начнем практику

Несколько оговорок перед началом

- Задавайте вопросы в процессе с помощью чата или же поднятой руки; блоки с Q&A сессиями в т.ч. будут
- Я использую ChatGPT как имя нарицательное, т.е. под этим я подразумеваю любой GenAI чат (deepseek, qwen, etc)
- Часть терминов (vibe/deep coding и т.д.) закрепились в комьюнити, но не являются формализованными инженерными понятиями
- Под AI IDE я подразумеваю 'LLM, интегрированную в среду разработки'

Структура занятия



ITMO | 2026

Давайте знакомиться



Владислав



Golang разработчик

в A7-tech (ex. RUTUBE)



Практик LLM-решений для кода

- Активно использую и тестирую все новые LLM-инструменты
- Верю, что будущее за инженерами, которые **правильно** используют AI IDE
- Я разработчик, а не менеджер, так что все презентации сгенерировал :)



Кейс #1: Deep coding на незнакомом ЯП

Срочная задача от менеджера: парсинг PDF-шаблонов

Проблема

- Срочно нужен парсинг PDF (как всегда, "еще вчера")
- Я Golang-разработчик, но подходящих библиотек на Go нет

Решение с AI IDE

- Выбираю незнакомый мне ЯП python, поскольку он идеален для решения моей задачи
- Постоянно задаю дополнительные вопросы по сгенерированному коду, тем самым изучая новый ЯП
- Закрыл "большую" задачу бизнеса за 1 день

Почему без AI IDE это было бы невозможно (качество кода, дедлайн)

- Копировать множество файлов (среда разработки ↔ ChatGPT) неудобно и медленно
- Многократная итерация генерации кода с запуском CLI тулов до тех пор, пока код не будет собираться/проходить тесты

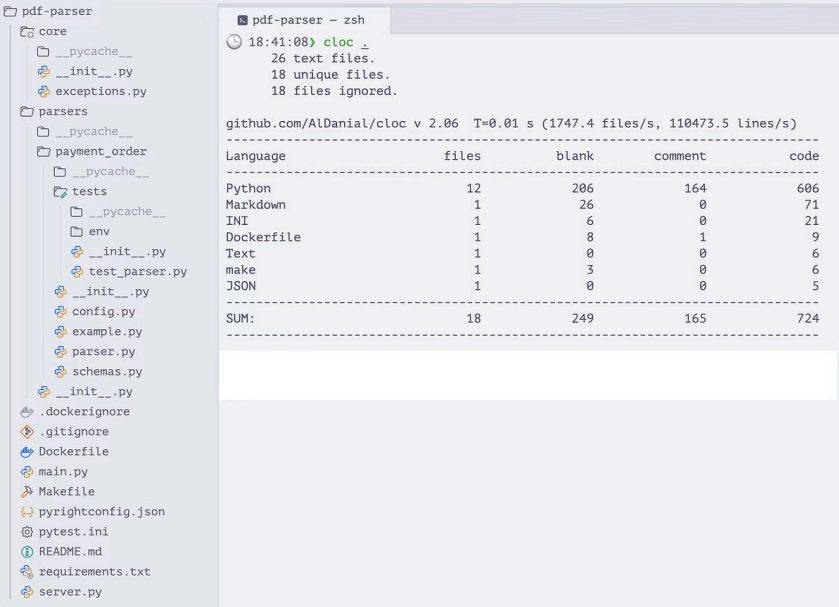
Выводы

AI IDE **позволяет:**

- выбирать ЯП исходя из пригодности под решение задачи, а не наших текущих навыков
- качественно решать в т.ч. рабочие задачи в очень сжатые сроки
- увеличивать плотность талантов за счёт усиления продуктивности опытных инженеров

При этом **требует:**

- дополнительно анализировать сгенерированный код, поскольку вся ответственность за результат лежит на разработчике



Кейс #2: Vibe coding для MVP

От идеи до успешного деплоя за 8 часов

Задача

- Создать лендинг и админку с целью валидации идеи

Подход

- 100% vibe coding — ни строчки кода вручную

Результат

- Работающий MVP для валидации идеи
- Не знаю, как все работает под капотом; сломается → не смогу починить самостоятельно

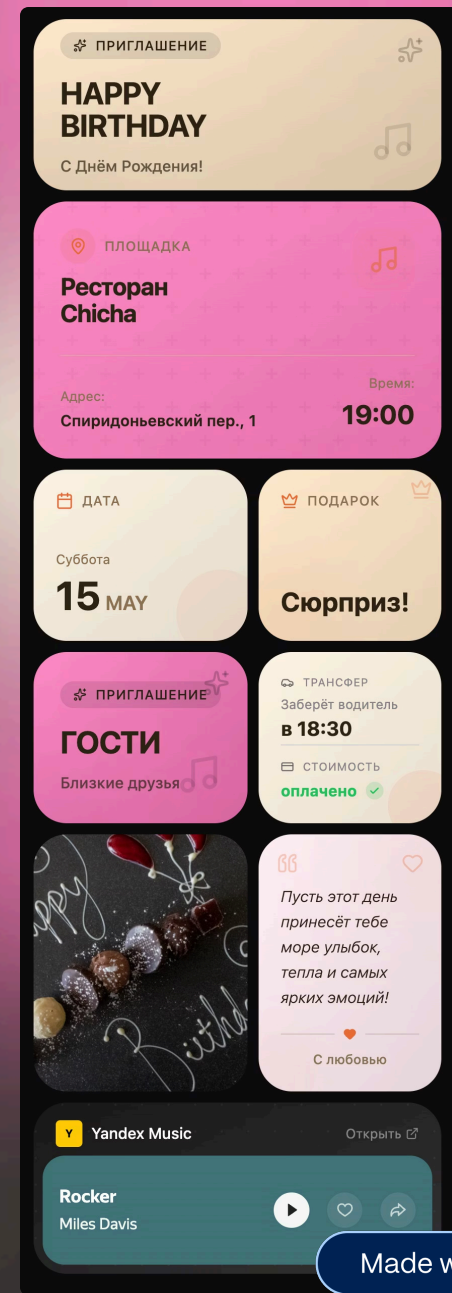
Дальнейшее развитие

Идея понравилась → переходим от vibe coding к deep coding, т.к. мы должны нести ответственность за результат

Вывод

Vibe coding идеален для быстрых экспериментов и MVP, но требует перехода к глубокому пониманию кодовой базы для продакшена

📄 Сайт доступен по gift-card.tech; **login/pass:** admin/pupupu



Кейс #3: Swarm coding — C-компилятор на Rust

16 агентов Claude, 2 недели, 100 000 строк кода — без участия человека

Задача

- Создать с нуля C-компилятор на Rust, способный скомпилировать ядро Linux

Подход

- 16 параллельных Claude Code работают над общей кодовой базой
- Каждый агент берёт «лок» на задачу, решает, пушит, берёт следующую
- ~2 000 сессий, ~\$20 000 за API

Результат

- Собирает Linux 6.9, FFmpeg, SQLite, Redis
- 99% pass rate на GCC torture test suite
- Компилирует и запускает Doom

Ограничения (честно)

- Код менее эффективен, чем GCC без оптимизаций
- Новые фишки периодически ломали существующий функционал

«AI хорошо умеет собирать известные техники и оптимизироваться под измеримые критерии. Но **плохо справляется с задачами, где нет чёткого критерия правильности** — а именно это и нужно для production-систем.»

— Chris Lattner, автор LLVM, Clang, Swift

Ключевой навык будущего — выстраивание среды, в которой ИИ не потеряется.

— Николас Карлини, Anthropic

Исходники: github.com/anthropics/claudes-c-compiler

Статья: anthropic.com/engineering/building-c-compiler



r/ClaudeAI • 2h ago
Prestigious-Use6804

Cuckcoding

Humor



↑ 438 ↓

17

2

Share

Report

Настройка окружения

01

Что понадобится

- VS Code + [Roo Code](#) (подробнее о нем ниже)
- Python (python --version)
- Артефакты прошлых практик (DoD, DoR, PlantUML)

03

Виртуальное окружение

- `python -m venv venv`
- `source venv/bin/activate`

05

Регистрация в OpenWeather

- Зарегистрируйтесь, получите api-ключ

02

Клонирование проекта

- Переместите все артефакты 2ой практики в `practice_03/docs`
- Откройте `practice_03` в новом окне

04

Подключение к API VSELLM

- `openai-compatible`
- `https://litellm.data-light.ru`
 - `enable-streamin:false`
 - будем использовать `claude-haiku-4.5`

Почему Roo Code?

Open-source AI расширение для VS Code

Работает без ограничений

Подключается к любому провайдеру, включая vsellm.ru без ограничений в РФ

Прозрачность

Нет индексации кодовой базы из коробки, как это есть в курсоре → лучше для обучения

Open-source

Код открыт, без подписок

VS Code

Самый популярный редактор :)

Базовые навыки переносимы в любую другую среду: Cursor, Claude Code, Opencode

Рассказываю на примере Roo Code, но вы вольны использовать другой инструмент.

Наставление перед реализацией 1го эндпоинта

AI IDE — быстрый джун, ответственность за результат на 100% лежит на вас



LLM слепа

Roo Code не индексирует кодовую базу автоматически



Ваша задача

Наполнить контекст релевантной информацией



Без контекста

Результат непредсказуем, агент выдумает своё

 **Важно:** Если агент начинает самостоятельно искать множество файлов → вы обогатили LLM недостаточным контекстом.

Реализация GET /weather/{city}

Интуитивный подход

01

Загрузите контекст

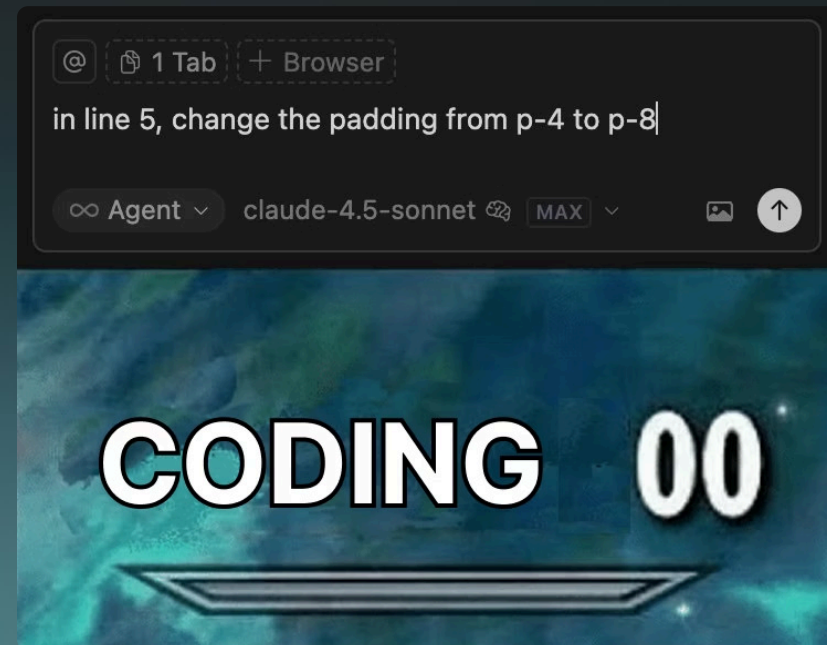
Не загружайте весь @docs, а лишь ту часть, которая относится к GET /weather/{city}.

02

Проверьте результат

Если не устраивает отсутствия констант, архитектура сервиса, используемые библиотеки т.д. - корректируйте агента в чате (в режиме Code)

- ❏ **Смешной, но работающий лайфхак:** Спросите LLM "Ты уверен на 100 из 100, что фича полностью реализована и работает корректно?"



Q&A TIME



.rooignore: контроль контекста

	
Проблема AI-агент видит весь проект — включая секреты, мусор и нерелевантные файлы. Это засоряет контекст, увеличивает стоимость запросов и создаёт риск утечки.	Решение Создать .rooignore в корне проекта — работает как .gitignore, но для Roo Code.

Пример .rooignore

```
# Секреты и окружение
.env
.env.*
*.pem
*.key
credentials.json

# Зависимости
node_modules/
__pycache__/*
*.pyc

# Сборка и артефакты
dist/
build/
*.log

# Не нужно AI
*.png
*.jpg
*.pdf
```

Реальные кейсы утечек

Knostic, 2025
Claude Code автоматически читает .env и отправляет содержимое на серверы
[Читать статью →](#)

TechCrunch, июль 2025
100 000 чатов ChatGPT с приватными данными проиндексированы Google
[Читать статью →](#)

Malwarebytes, январь 2026
300 млн сообщений AI-чатов утекли из-за открытой Firebase
[Читать статью →](#)

Rules: решение недетерминированности



Проблема

GenAI не детерминирован, т.е. на 100 одинаковых запросов мы получим 100 разных ответов.



Решение

Создать `.roo/rules.md` со стайл-гайдом проекта.

Пример правил

- Используй `async/await` для всех I/O операций
- Все ошибки через `HTTPException`
- Type hints обязательны
- Pydantic v2 для валидации
- Оставляй аннотацию только в неочевидных местах

Modes: разные режимы работы



ask

Режим без доступа к редактированию
Только отвечает на вопросы в чате



code

Дефолтный режим для разработки
Может читать и редактировать файлы



architect

Режим планирования и критики
Пишет подробный план, критикует решения

💡 Связь с RCTF

Каждый режим — это уже частично заполненный RCTF-промпт: Role (архитектор / кодер / ассистент) и Format (план vs код vs ответ в чате) заданы за вас. Вам остаётся сформулировать Context и Task.

⚡ Стратегия двух моделей: думает умная, делает быстрая

Планирование — задача на рассуждение, а реализация — на следование инструкциям. План — это текстовый артефакт, который передаёт «интеллект» сильной модели в контекст слабой.

- Architect mode → сильная модель (Opus, DeepSeek-R1..): декомпозиция, edge-cases, архитектурные решения. ~1500 токенов output.
- Code mode → быстрая модель (Sonnet, GPT-4o, Gemini Flash): реализация по готовому плану. ~8000 токенов output.
- Экономия ~70% при том же качестве архитектуры: план = 10% токенов, реализация = 90%. Sonnet с хорошим планом практически не уступает Opus.



Всегда проводи ревью плана перед передачей — иначе быстрая модель аккуратно реализует неправильное решение.



Вы можете создать свой режим с набором прав (read, write), MCP (об этом в следующей лекции) и системным промптом

Multi-chat workflow

"Начинай новый чат, когда текущий стал шумным (>80k токенов), произошла смена роли, или модель начала «плыть»."

Одного чата достаточно

- Компактная задача (< 30k токенов контекста)
- Одна роль (например, только code)
- Нет смены парадигмы (рефакторинг, не архитектура)

Нужен новый чат

- Контекст раздут (50-80k+ токенов)
- Смена роли (architect → code)
- Модель начала повторяться или "плыть"
- Переход от ревью к реализации

Пример: POST /subscribe (один из вариантов)

Чат 1: architect

Критика архитектуры + план реализации (если контекст позволяет, можно в одном чате)

Чат 2: code

Реализация по плану. Новый чат — контекст чистый, только план и код.

Это не догма. Если задача простая и контекст чистый — можно обойтись одним чатом. Главное — следить за качеством ответов.

Auto-approve: удобство и безопасность

Проблема

Агент часто запрашивает approve для типовых операций → теряем скорость/комфорт совместной разработки с агентом.

Решение

Настроить auto-approve для шаблонных операций

Важные ограничения

- Deny-list для опасных команд и паттернов (например, `rm -rf`)
- Запрет read/write вне корня проекта

Реализация POST /subscribe

Продвинутый подход: rules, modes, multi-chat

- 1 — Создать `.rooignore`
- 2 — Создать `.roo/rules.md`
- 3 — Использовать разные режимы (architect, code)
- 4 — Использовать multi-chat workflow (1 чат = 1 задача)
- 5 — Настроить auto-approve

📌 Продвинутый подход: Rules + modes + multi-chat = более предсказуемый и качественный результат

Три подхода к разработке



No LLM coding

Классическая разработка без AI-ассистентов

- Полный ручной контроль
- Больше времени на рутину



Deep coding

Вдумчивая работа инженера-разработчика над production-ready решениями

- Понимание каждой строчки кода
- Полная ответственность за результат



Vibe coding

Быстрая демонстрация идей, прототипирование, обучение

- Скорость важнее реализации



Запомните 2 правила:

- всегда коммитьте изменения в Git до, после и во время работы с LLM
- vibe coding применим для обучения и прототипов, но не для production-разработки

Зачем программировать в AI IDE?

Работа без копипаста между чатом и IDE

AI IDE сама читает и создаёт файлы, понимает структуру проекта и предоставляет удобный diff.

Меньше ручных действий → быстрее → меньше ошибок.

Идиоматичный код под конкретный проект

Правила в .goos настраиваются под каждый репозиторий. LLM генерирует идиоматичный код каждого из наших проектов.

Постоянная память

Автоматическое формирование memory bank'a (MCP).

Код сразу проходит проверку

После изменений автоматически запускаются наши линтеры, тесты и т.д. AI встраивается в цикл написал → проверил → поправил.

Масштабирование инженерных практик

Подходы к тестам и изменениям масштабируются от одного разработчика на всю команду.

Юзкейсы из жизни разработчика

Решение merge-конфликтов, review коммитов/MR'ов, анализ staged/unstaged изменений.



AI IDE — это не «чат с кодом», а инструмент, встроенный в инженерный процесс.

Как программировать в AI IDE?

Чеклист для эффективной работы

Адаптация и понимание границ

Настройте инструменты под себя, изучите возможности и ограничения модели

Только после адаптации можно уверенно использовать AI coding

@Андрей Карпаты

Гигиена контекста

Качество моделей падает после 70-80k токенов → пропуски логики и галлюцинации

Используйте субагенты и разделяйте задачи на меньшие контексты

Детерминированный Feedback Loop

Никакого слепого доверия модели — агент должен проверять себя

- Пайплайн: Код → Линтер/Type-check/Тесты → Авто-фикс агентом
- Без этого генерируется легаси-код

Приоритет планирования (правило 90/10)

Генерация кода без плана = сжигание бюджета

- ~2 часа на архитектуру с рассуждающими моделями и только после реализация, поскольку плохой план кратно увеличивает расход токенов

Обучение через AI

- Включайте режим объяснений, чтобы понимать причины изменений
- Просите визуализации (ASCII-диаграммы, HTML-презентации) для незнакомого кода
- Объясняйте своё понимание AI, получайте уточняющие вопросы (метод утёнка)

Эволюция правил (.roo/AGENTS.md)

- Обновляйте правила после каждой ошибки: "обнови CLAUDE.md, чтобы не повторять это"
- Правила должны эволюционировать вместе с проектом
- Для каждого отдельного проекта отдельный .roo, можете наследоваться от базового .roo

Будущее за AI-first средами

JetBrains предоставляет бесплатный доступ ко всем продуктам компаниям взамен на сбор данных для обучения AI моделей

~\$1K

Экономия в год

На лицензиях JetBrains

📌 **Вывод:** JetBrains отдаёт лицензии бесплатно сегодня, чтобы завтра иметь преимущество в гонке за AI-инструменты.

Основные игроки рынка AI IDE

Zed

AI-ориентированный open-source редактор на Rust. Быстрый, минималистичный; часто инженеры переходят из neovim.

Claude Code

CLI-agent в терминале. Не принуждает менять редактор. Rewind, хуки, маркетплейс скилов, ACP-интеграция, app.

Cursor

Fork VS Code. Полностью AI-first, совместим с маркетплейсом, лучший inline-autocomplete, своя модель composer. Де-факто стандарт индустрии.

Roo Code

Open source расширение для тех, кто не хочет покидать VS Code.

Warp

Первая в мире ADE (Agentic Development Environment), terminal-first подход.

Opencode

Бесплатный аналог claude code, нужен API LLM.

Summary

Ответственность

Вы несёте 100% ответственность за сгенерированный код

Понимайте каждую строчку

Версионирование

Постоянно коммитить результаты работы LLM

Работать с Git обязательно

Ваша ценность

Знание кодовой базы проекта

Быстрое решение инцидентов и оценка решений

Домашнее задание

Основные задания

1	2
Реализовать 2 эндпоинта (4 балла) - DELETE /subscribe/{id} - GET /subscriptions Главное: экспериментируйте с LLM в процессе.	Создать ≥3 полезных мода в Roo Code (2 балла) Главное: агенты должны иметь смысл (например, для тестирования, анализа текущей архитектуры, qa notes и т.д.), в дальнейшем они будут автономно вызываться в режиме оркестрации.
3	4
Мигрировать правила (1 балл) Ознакомьтесь с agents.md, мигрируйте правила из .foo в agents.md (универсальный формат).	Рефлексия в свободном формате (1 балл) Отрефлексируйте процесс вайб-кодинга (ДЗ) и/или реализацию 2 обязательных эндпоинтов на практике, подготовьте вопросы к следующей практике по AI-IDE. p.s. task.py не используйте; рефлексию предоставьте в .md файле в свободном формате



Навайбкодить что угодно на новом ЯП (+4 балла)

Лендинг/сайт (рекомендую [VO](#)), CLI тулза, либа на Rust для Python и т.д.

Проверка по записи экрана с демо или же онлайн-демо на защите.

p.s. попробуйте использовать [handy](#) (транскрибация голоса), чтобы быть истинными вайбкодерами!

Всего: 12 баллов



 Используйте API ключ от [vsellm.ru](#) — токены конечны, расходуйте разумно

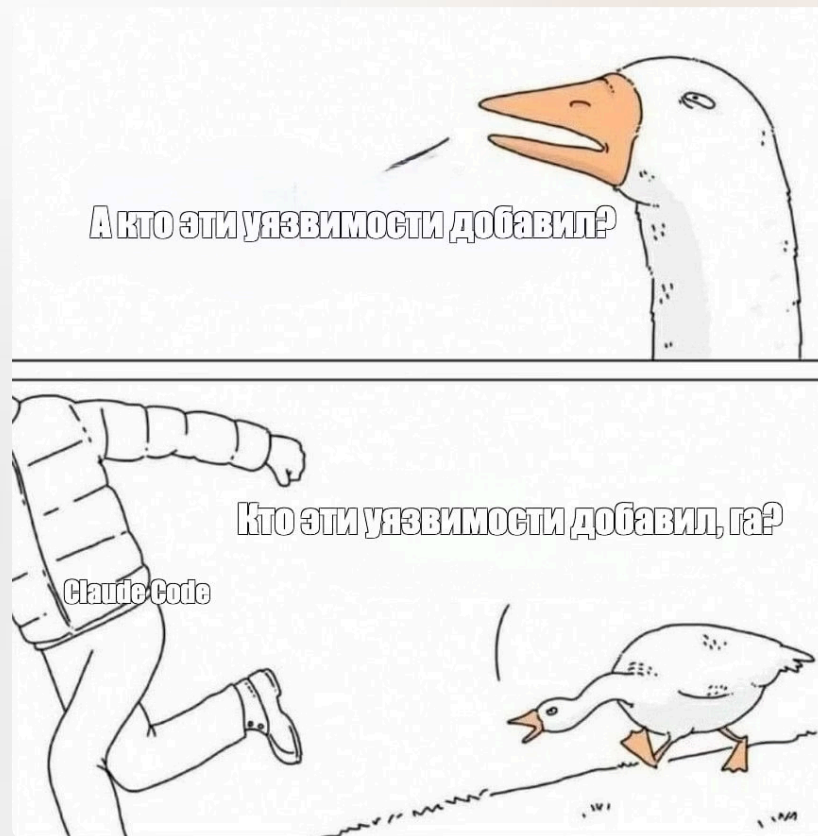
Q&A

Время для вопросов

Задавайте вопросы любой сложности, все обсудим.

Ниже пример для вашего вдохновения:

- ☐ Как оплачивать LLM? API-based pay vs subscriptions? Как сэкономить на оплате?
- ☐ Возможно ли бесплатно использовать AI IDE?
- ☐ Что такое контекст? А как расшифровывается LLM?
- ☐ Каких LLM-провайдеров стоит выбрать? Насколько хороши китайские модели? ([GLM 5.0](#), например)
- ☐ Как работодатели относятся к использованию LLM, интегрированных в кодовую базу?
 - ☐ Как не нарушать 152-ФЗ? (закон о передаче перс. данных граждан РФ за границу)
- ☐ Я не понял, какое ДЗ?



Темы следующей практики

MCP (Context7, Memory-Bank и т.д.)

Code-review в GitHub

Оркестрация параллельных агентов

Спасибо за внимание!

Telegram: [@NOgameNo1fee](https://t.me/@NOgameNo1fee)

Перечень упомянутых ссылок и доп. ресурсов доступны в [NotebookLM](#)

ITMO | 2026



**Поделитесь обратной
связью по занятию**